

Visão Computacional com CNNs e Transformers — Aula 3

BERT e Modelos Encoder-Only: Atenção Bidirecional Profunda, Pré-Treinamento (MLM & NSP) e Cabeças Plugáveis

Etapa / Aula 03

Prof. Allan Spadini • Pós-Graduação em Inteligência Artificial & Visão Computacional

✦ A Cegueira Unidirecional Causal

✦ Taxonomia Transformer: Encoder vs Decoder

✦ Anatomia Conceitual do BERT & StatQuest Flow

✦ O Superpoder das Cabeças Plugáveis

✦ Embeddings Triplos (Token, Segment, Position)

✦ Masked Language Model (MLM) & A Regra 80/10/10

✦ Next Sentence Prediction (NSP) & Coerência

✦ Fine-Tuning de Ponta a Ponta vs Feature Extraction

✦ Casos de Uso: Sentimento, NER (-100), SQuAD & SBERT

✦ Evolução do BERT: RoBERTa, DistilBERT & DeBERTa

A Situação-Problema: A Cegueira Unidirecional

Por que Modelos Causais (GPT e LSTMs) Falham na Compreensão Holística de Contexto

Paradigma de Atenção:

← Unidirecional Causal (GPT / RNN)

↔ Bidirecional Profundo (BERT)

✓ Contexto Total: 'banco' conecta-se diretamente com 'bloqueou' e 'cartão'!

SEQUÊNCIA DE TOKENS DE ENTRADA:



↔ Matriz de Atenção Plena: Todos os tokens consultam todos os tokens ($O(L^2)$)

$h_{\text{banco}} = f(\text{O}, \text{gerente}, \text{do}, \text{banco}, \text{bloqueou}, \text{meu}, \text{cartão})$

Desambiguação perfeita: Instituição Financeira / Cartão de Crédito!

Formulação Causal (Decoder - GPT):

$P(w_t \mid w_1, \dots, w_{t-1}) \implies$ Atenção Triangular Inferior com $-\infty$

Formulação Bidirecional (Encoder - BERT):

$h_t = \text{Encoder}(w_1, \dots, w_L)_t \implies$
Atenção Cruzada Completa em Todas as Camadas

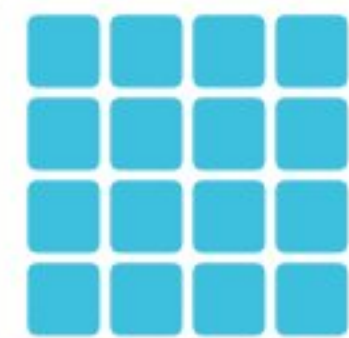
A Família Transformer: A Bifurcação das Arquiteturas

Encoder-Only (BERT) vs Decoder-Only (GPT) vs Encoder-Decoder (T5)

ENCODER-ONLY

FOCO DA AULA 3

BERT, RoBERTa, DeBERTa, ViT



Atenção Bidirecional Plena
Matriz Completa (Todos vêem todos)

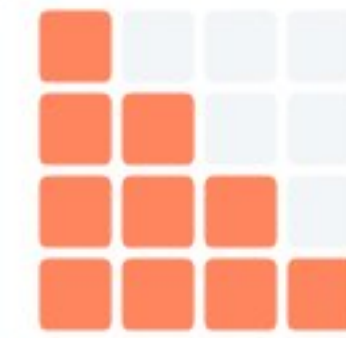
Aplicações Típicas:

- Classificação de Sentimento & Tópico
- Reconhecimento de Entidades (NER)
- Question Answering Extrativo (SQuAD)
- Busca Semântica & Embeddings Densos
- Visão Computacional (Vision Transformer - ViT)

Saída Principal: Representações contextuais ricas de cada token (h_i) e da sequência inteira ($h_{[CLS]}$).

DECODER-ONLY

GPT-2/3/4, LLaMA, Claude, Mistral



Atenção Causal Mascarada
Matriz Triangular Inferior (Futuro = $-\infty$)

Aplicações Típicas:

- Geração Autorregressiva de Texto
- Assistentes Conversacionais (Chatbots)
- Geração e Completude de Código
- Raciocínio Passo a Passo (CoT)

Saída Principal: Distribuição de probabilidade sobre o vocabulário para prever o próximo token $P(w_t | w_{<t})$.

ENCODER-DECODER

Vaswani et al., T5, BART, mT5



Atenção Bidirecional + Cross-Attention
Encoder Pleno + Decoder Causal + Cross

Aplicações Típicas:

- Tradução de Idiomas (Machine Translation)
- Sumarização Abstrativa de Documentos
- Conversão Texto-para-Código / SQL
- Modelagem Sequence-to-Sequence (Seq2Seq)

Saída Principal: Mapeia sequência de comprimento N para outra sequência de comprimento M arbitrário.

💡 **Por que o BERT é Encoder-Only?** Porque tarefas de compreensão de texto (sentimento, busca, entidades) não precisam gerar palavras futuras uma a uma. Elas necessitam de *visão panorâmica de 360 graus* sobre toda a frase de uma só vez!

A Anatomia da Arquitetura do BERT: Da Palavra à Decisão

Tokens, Projeções Lineares, Atenção Q/V/K, Context-Aware Embeddings e o Acoplamento de Cabeça

Destaque Arquitetural:

Visão Completa

1. Word Embeddings

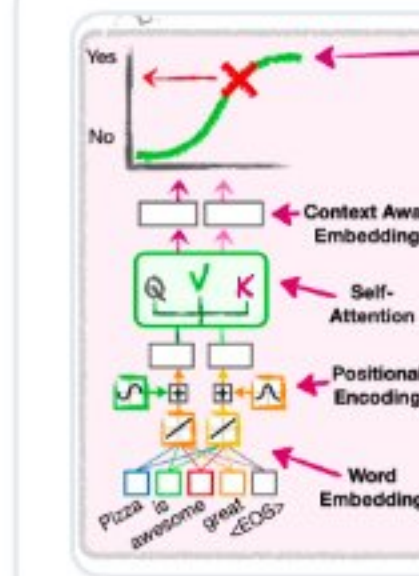
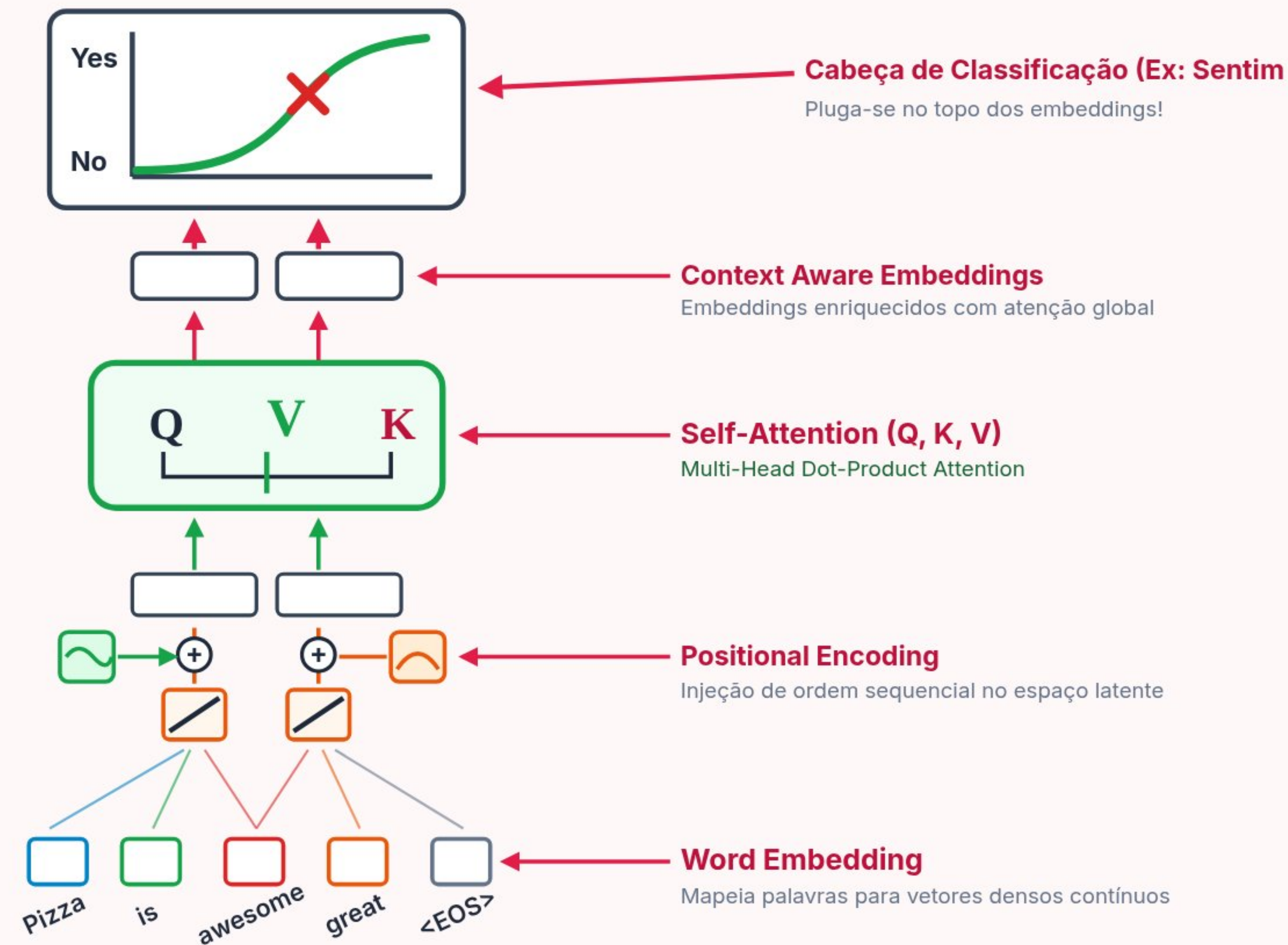
2. Positional Encoding (+)

3. Self-Attention (Q, K, V)

4. Context Aware Embeddings

5. Cabeça de Classificação (Yes/No)

Arquitetura Didática Modular: Tokens $\rightarrow$ Q,V,K $\rightarrow$ Classificação



O Diagrama Mental Canônico

Mostra a essência do Encoder: uma pilha vertical onde a entrada é contextualizada por *Self-Attention* e o topo conecta-se a **qualquer função de perda** (Sigmoid, Softmax, Regressão linear).

Tensores ao Longo da Pilha (BERT-Base):

- **Entrada de Tokens:** $[B, L]$ inteiros de 0 a 30.521.
- **Embeddings + Posição:** $[B, L, 768]$.
- **Projeções Q, K, V:** $[B, h=12, L, d_k=64]$.
- **Context Aware (H):** $[B, L, 768]$ em todas as 12 camadas.
- **Saída da Cabeça:** $[B, K]$ logits de classificação.

O Segredo: A Saída é Totalmente Desacoplada!

O corpo do BERT aprende a semântica da linguagem. A cabeça de saída no topo (o gráfico Yes/No) é apenas uma pequena camada linear substituível!

O Conceito Chave: Cabeças Plugáveis (Task-Specific Heads)

Um Único Backbone Pré-Treinado, Múltiplas Aplicações de Mercado

1. Classificação de Sentença

Análise de Sentimento / Spam / Tópicos

2. Classificação de Tokens (NER)

Extração de Entidades Nomeadas & POS Tagging

3. Question Answering Extrativo

SQuAD: Localização de Resposta em Texto

4. Par de Sentenças (NLI)

Inferência Natural: Entailment vs Contradição

CABEÇA CONECTADA: 1. CLASSIFICAÇÃO DE SENTENÇA

$\text{Linear}(768 \rightarrow K) + \text{Softmax}$

Probabilidade de cada classe (Positivo / Negativo / Neutro)

Interface de Tensores Ocultos $[B, L, 768]$

BONE BERT PRÉ-TREINADO (CONGELADO OU FINE-TUNED)

12 Camadas Transformer Encoder • 12 Attention Heads • $H = 768$ • 110M Parâmetros

Self-Attention Bidirecional → LayerNorm → Feed-Forward (MLP 4×) → LayerNorm

[CLS]

token_1

token_2

token_3

[SEP]

MODALIDADE SELECIONADA

1. Classificação de Sentença

Caso de Uso: Análise de Sentimento / Spam / Tópicos

Tensor de Entrada:

$h_{[CLS]} \in \mathbb{R}^{768}$

Arquitetura da Cabeça:

$\text{Linear}(768 \rightarrow K) + \text{Softmax}$

Exemplo Prático:

[CLS] Este restaurante superou todas as expectativas! [SEP] → Positivo (99.2%)



Laboratório 1: Simulador de Cabeças Plugáveis do BERT

Alterne Entre as 4 Tarefas Canônicas e Inspeção os Tensores Conectados ao Topo

1. Classificação Sentimento
2. Extração Entidades (NER)
3. QA Extrativo (SQuAD)
4. Par de Sentenças (NLI)

CABEÇA MODULAR CONECTADA AO TOPO

$W \in \mathbb{R}^3 \times 768, b \in \mathbb{R}^3$

nn.Linear(in_features=768, out_features=3) + Softmax

Vetor h_[CLS] (índice 0) condensa toda a oração

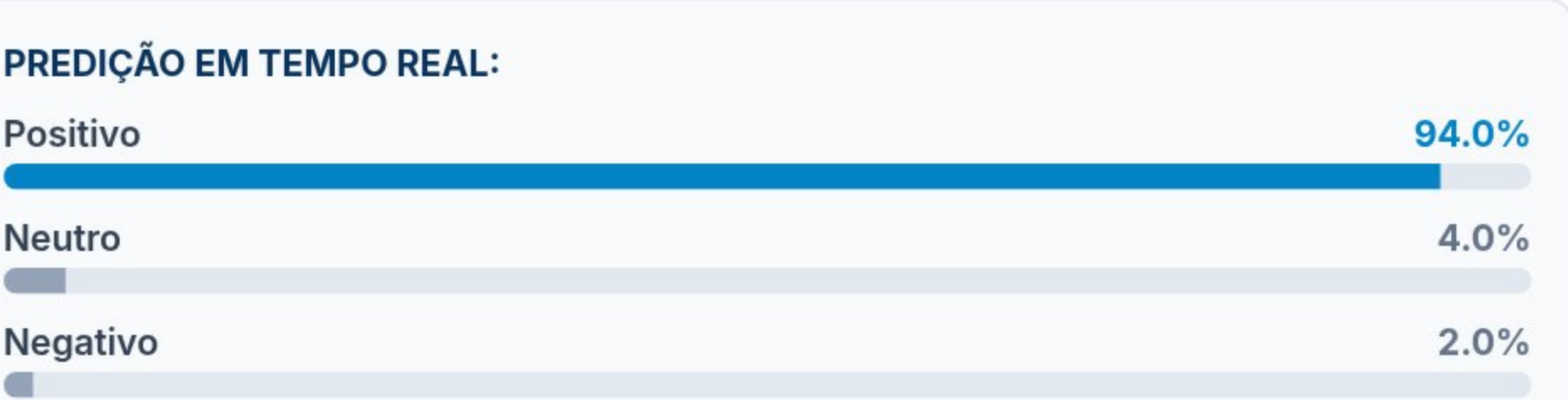
▲ ▲ ▲ Tensores Ocultos Contextuais [Batch, L, 768] ▲ ▲ ▲

BACKBONE BERT PRÉ-TREINADO (110M PARÂMETROS)

12 Camadas de Atenção Bidirecional 360° • Sem máscaras causais • Pesos Universais

ENTRADA FORMATADA COM TOKENS ESPECIAIS:

[CLS] O novo algoritmo superou todas as metas da equipe! [SEP]



💡 **Conclusão Técnica:** Note como o *mesmo* modelo BERT serve para 4 tarefas radicalmente distintas. Mudamos apenas a camada final de projeção e a função de custo!

A Estrutura de Embeddings Tripla do BERT

A Soma Vetorial: Token Embeddings + Segment Embeddings + Position Embeddings

Equação de Entrada do BERT: $E_i = \text{TokenEmbed}(w_i) + \text{SegmentEmbed}(s_i) + \text{PositionEmbed}(i)$

Dimensão de cada vetor: $d_{\text{model}} = 768$

Token Embeddings:

[CLS]

meu

cão

é

fofo

[SEP]

ele

gosta

de

brincar

[SEP]

+

Segment Embeddings:

E_A

E_A

E_A

E_A

E_A

E_A

E_B

E_B

E_B

E_B

E_B

+

Position Embeddings:

pos_0

pos_1

pos_2

pos_3

pos_4

pos_5

pos_6

pos_7

pos_8

pos_9

pos_10

=

Entrada Camada 1:

E_0

E_1

E_2

E_3

E_4

E_5

E_6

E_7

E_8

E_9

E_10

Token WordPiece: Vocabulário de 30.522 subpalavras. O token [CLS] inicia toda sequência e [SEP] separa frases.

Segment IDs (0 ou 1): Informa à rede se o token pertence à Sentença A ou à Sentença B, crucial para pares de sentenças (NSP e QA).

Position Apreendida: Diferente de Vaswani et al. (senos/cossenos fixos), no BERT cada posição de 0 a 511 tem seu próprio vetor treinável de 768 dimensões!

O Pré-Treinamento 1: Masked Language Model (MLM)

A Tarefa Cloze, a Perda Cross-Entropy e a Regra de Ouro 80 / 10 / 10 da Google AI

Objetivo Auto-Supervisionado:

Fluxo do MLM & Projeção Vocab

A Regra 80 / 10 / 10 da Google AI

Perda aplicada estritamente sobre os 15% de tokens mascarados



Função de Perda Cross-Entropy do MLM:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i = w_i^* | \tilde{X})$$

Onde $\mathcal{M}$ é o conjunto das posições mascaradas e w_i^* é a palavra real original do texto.

🎯 Por que o MLM Revolucionou o Pré-Treino?

Se usássemos atenção bidirecional comum para prever a próxima palavra, a rede simplesmente *copiaria* a palavra que já está visível à frente. Ao substituir a palavra por [MASK], a rede é obrigada a deduzir a semântica a partir do contexto circundante!

⚡ **Tied Weights (Amarração de Pesos):** A matriz da cabeça linear final W_{vocab} usa os mesmos pesos transpostos da matriz de Word Embeddings da entrada, economizando 23.4 milhões de parâmetros!

Laboratório 2: Simulador do Masked Language Model (MLM)

Mascare Tokens em Tempo Real e Veja a Projeção de Softmax Sobre o Vocabulário

Selecione uma Frase para Mascaramento:

Frase 1

Frase 2

Frase 3

Vocabulário Total: 30.522 tokens WordPiece

CLIQUE NA PALAVRA DESTACADA PARA INSPECIONAR A PREVISÃO:

[CLS]

O

gerente

do

[MASK]

bloqueou

o

cartão

do

cliente

[SEP]

Como o BERT resolveu este [MASK]?

A palavra "bloqueou" e "cartão" à direita + "gerente" à esquerda forçam a interpretação financeira!

Perda Cross-Entropy nesta posição:

$$\mathcal{L}_i = -\log(0.88) = 0.1278$$

✓ Alvo Reconstruído!

TOP-5 PALAVRAS PREVISTAS PELO BERT:

Softmax(z)

1. "banco" ✓ (Alvo Real) 88.0%

2. "sistema" 7.0%

3. "aplicativo" 3.0%

4. "computador" 1.0%

5. "caixa" 1.0%

⚡ Dimensão da Saída: O vetor $h_{[MASK]}$ de 768 números foi multiplicado pela matriz de 30.522 linhas para gerar a pontuação de cada palavra do dicionário!

O Pré-Treinamento 2: Next Sentence Prediction (NSP)

Compreensão de Relacionamentos Inter-Sentenciais: IsNext (50%) vs NotNext (50%)

Exemplo de Treinamento:

✓ 50% dos Pares: IsNext (Sequência Real)

✗ 50% dos Pares: NotNext (Frase Aleatória)

Classificação binária calculada **exclusivamente** sobre o vetor $h_{[CLS]}$

PREDIÇÃO: IsNext (1)

Confiança: 98.7% (Continuidade)

(Tokens de palavras h_i são usados pelo MLM, não pelo NSP)

$W_{nsp} \cdot h_{[CLS]}$

BERT ENCODER (Atenção Bidirecional Cruza Sentença A e Sentença B)

[CLS]

Sentença A: "O homem foi à padaria."

[SEP]

Sentença B: "Comprou 2 pães."

Segment ID: 0 (Sentença A)

Segment ID: 1 (Sentença B)

Formulação Matemática do NSP:

$$P(\text{IsNext}) = \text{Softmax}(W_{\text{NSP}} \cdot h_{[CLS]} + b)$$

$$\mathcal{L}_{\text{NSP}} = -y \log p - (1 - y) \log(1 - p)$$

Onde $y \in \{0, 1\}$ indica se a Sentença B realmente sucedia a Sentença A no corpus original.

Por que o NSP é Essencial?

O MLM ensina o modelo a entender o relacionamento entre palavras *dentro* de uma frase. O NSP ensina o modelo a compreender a coerência e lógica *entre duas frases distintas* (fundamental para QA e NLI).

Nota Histórica (RoBERTa): Em 2019, o Facebook AI demonstrou que removendo o NSP e treinando com blocos de texto contíguos de 512 tokens completos, o modelo mantinha ou até melhorava a precisão!



Laboratório 3: Inspetor de Pares e NSP

Inspecione a Coerência Semântica e a Decisão Binária no Vetor [CLS]

Selecione um Par de Sentenças:

Par 1 (IsNext)

Par 2 (NotNext)

Par 3 (IsNext)

Objetivo: Treinar compreensão de relacionamentos entre múltiplas frases

SENTENÇA A (Segment ID = 0)

"Alan Turing propôs o conceito da máquina universal."

Tokens 1 a N

[SEP] (Divisor de Sequências)

SENTENÇA B (Segment ID = 1)

"Esse modelo teórico definiu os fundamentos da computação moderna."

Tokens N+2 a M

Formato: [CLS] Sentença A [SEP] Sentença B [SEP]

CLASSIFICADOR BINÁRIO NSP:

IsNext (Continuação Real):

98.5%

NotNext (Frase Aleatória):

1.5%

Veredito do Modelo:

Forte coesão anafórica: "Esse modelo teórico" conecta-se diretamente com "máquina universal".

📌 **Vetor de Decisão:** Toda essa inferência é calculada com base única e exclusiva no vetor $h_{[CLS]}$!



A Função de Custo Conjunta e Especificações Oficiais

Otimização Multitarefa: $L_{total} = L_{MLM} + L_{NSP}$ e a Comparação BERT-Base vs Large

OBJETIVO CONJUNTO DE PRÉ-TREINAMENTO: $\mathcal{L}_{total} = \mathcal{L}_{MLM} + \mathcal{L}_{NSP}$

Otimizador: AdamW • Warmup linear nos primeiros 10.000 steps

Especificações Arquiteturais das Variantes Oficiais:

Hiperparâmetro	BERT-Base	BERT-Large
Camadas Encoder (L)	12	24
Dimensão Oculta (H / d_model)	768	1024
Attention Heads (A)	12 (d_k = 64)	16 (d_k = 64)
Feed-Forward Interno (4H)	3072	4096
Ativação Não Linear	GELU	GELU
Total de Parâmetros	110 Milhões	340 Milhões

Regra de Memória: O BERT-Base roda confortavelmente em GPUs intermediárias (8GB a 16GB VRAM). O BERT-Large exige ao menos 24GB VRAM (ou técnicas como Gradient Accumulation e FP16).

Corpus de Pré-Treinamento (3.3B Palavras):

- **BookCorpus:** 800 milhões de palavras extraídas de livros não publicados (ideal para capturar narrativa contínua e long-range context).
- **English Wikipedia:** 2.500 milhões de palavras (artigos informativos enciclopédicos completos).

Escala Computacional Google:

- **BERT-Base:** 16 Cloud TPUs (64 chips) por 4 dias contínuos.
- **BERT-Large:** 64 Cloud TPUs (256 chips) por 4 dias contínuos.
- **Batch Size:** 256 sequências de 512 tokens = 128.000 tokens por passo, totalizando 1 milhão de passos de gradiente.

O Grande Benefício para a Indústria: O Google gastou milhares de dólares em TPUs para pré-treinar o BERT. Nós podemos fazer *Fine-Tuning* para o nosso negócio em apenas alguns minutos no Google Colab!

Estratégias de Aplicação: Fine-Tuning vs Feature Extraction

Quando Descongelar o Backbone Inteiro e Quando Usar o BERT como Extrator Estático



Opção A: Fine-Tuning de Ponta a Ponta

Recomendado (Maior Acurácia)

- Todos os 110M parâmetros do BERT permanecem destravados (`requires_grad = True`).
- Os gradientes da perda da sua tarefa fluem por todas as 12 camadas.
- Exige taxa de aprendizado muito baixa: entre $2e-5$ e $5e-5$ para não destruir o pré-treino (catastrophic forgetting).
- Convergência ultrarrápida: geralmente apenas 2 a 4 épocas de treino.
- Atinge a acurácia máxima em benchmarks como GLUE, SQuAD e SST-2.

🚀 Padrão da indústria para alcançar o Estado da Arte (SOTA).



Opção B: Feature Extraction (Backbone Congelado)

Econômico (Menos VRAM)

- O backbone do BERT é 100% congelado (`requires_grad = False`).
- Atua como um conversor estático: texto bruto → embedding vetorial de 768 dimensões.
- Treina-se apenas um classificador tradicional leve no topo: Regressão Logística, SVM, Random Forest ou MLP.
- Custo computacional irrisório: os embeddings podem ser pré-calculados e cacheados em disco uma única vez.
- Excelente para cenários de pouquíssimos dados anotados ou GPUs limitadas.

🚀 Ideal para prototipação rápida e bancos de dados vetoriais.

⚡💡 Regra de Ouro: Com GPU disponível e foco em performance máxima, adote SEMPRE o Fine-Tuning de ponta a ponta com AdamW e $lr = 2e-5$.

Aplicação 1: Classificação de Sequências de Texto

Análise de Sentimentos, Detecção de Fraude e Tópicos com Pooling do Vetor [CLS]

Exemplo de Avaliação de Mercado:

Avaliação 1 (Positivo)

Avaliação 2 (Negativo)

Mapeamento canônico: $h_{[CLS]} \rightarrow \text{Dropout} \rightarrow \text{Linear}(768, K) \rightarrow \text{Softmax}$

PREDIÇÃO: Positivo (98.8%)

$\text{Softmax}(z) \rightarrow$ Distribuição sobre $K=3$ classes

nn.Linear(768, num_labels=3)
+ Dropout(p=0.1)

Pooling: apenas o índice $[:, 0, :]$

BERT ENCODER (AutoModelForSequenceClassification)

Todos os tokens comunicam-se bidirecionalmente • Vetor [CLS] condensa o sentido global

[CLS]

"O serviço de atendimento ao cliente resolveu meu pro..."

[SEP]

Rastreamento de Tensores PyTorch:

- `input_ids`: [Batch, SeqLen=128]
- `attention_mask`: [Batch, SeqLen=128]
- `last_hidden_state`: [Batch, SeqLen, 768]
- `cls_rep = h[:, 0, :]`: [Batch, 768]
- `logits = W(cls_rep)`: [Batch, num_classes=3]

Distribuição Softmax Prevista:

Positivo:	98.8%
Neutro:	0.8%
Negativo:	0.4%

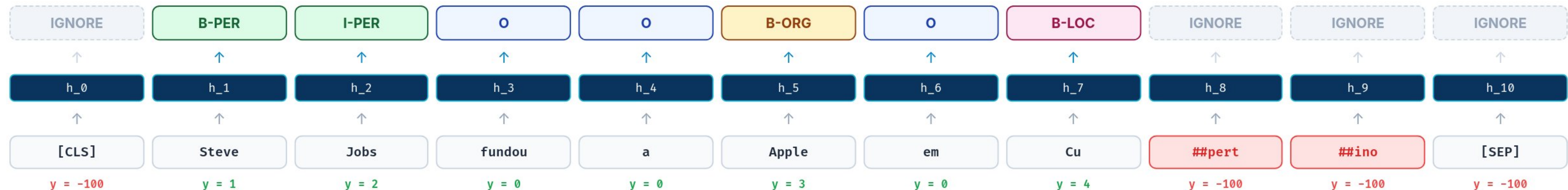
Função de Perda no Treino: $\mathcal{L} = \text{CrossEntropyLoss}(\text{logits}, y)$

Aplicação 2: Classificação de Tokens (NER)

Reconhecimento de Entidades e o Desafio Crítico de Alinhamento de Subwords com o Rótulo -100

O Desafio de Engenharia do NER com WordPiece: A Máscara do Rótulo Especial -100 no PyTorch

`nn.CrossEntropyLoss(ignore_index=-100)`



Por que usar $y = -100$ nas Subwords seguintes?

A palavra "Cupertino" foi fragmentada pelo tokenizer WordPiece em 3 pedaços: ["Cu", "##pert", "##ino"]. Se atribuíssemos B-LOC a todos os 3 pedaços, o modelo preveria 3 cidades separadas. Ao rotular apenas a 1ª subword com B-LOC e as subsequentes com -100, o gradiente não é computado nos fragmentos e a entidade é preservada como uma unidade inteira!

Tensor de Saída PyTorch:

```
logits = Linear(768, num_tags=9)(h)
Shape: [Batch, SeqLen=11, num_tags=9]
```



Aplicação 3: Question Answering Extrativo (SQuAD)

Localização Precisa de Respostas em Documentos: Os Vetores de Start e End Logits

Mecanismo Extrativo de Span (SQuAD): $s_i = S \cdot h_i, \quad e_j = E \cdot h_j \implies \text{Span} = \arg \max_{j \geq i} (s_i + e_j)$

Vetores Treináveis: $S, E \in \mathbb{R}^{768}$

PERGUNTA: "Qual monumento foi inaugurado em 1889?"

Extrativo vs Generativo:

Diferente de um modelo GPT que gera texto livre (podendo alucinar), o BERT é **100% fiel ao documento original**: ele apenas aponta os ponteiros de início e término no texto já existente.

▲ Probabilidade de INÍCIO (Start Logits via Vetor S)

▼ Probabilidade de FIM (End Logits via Vetor E)



Formatação da Entrada com Segment IDs:

[CLS] Pergunta [SEP] Contexto [SEP]
Segment ID A = 0 (Pergunta)
Segment ID B = 1 (Contexto)

RESPOSTA EXTRATIVA EXTRAÍDA:

"Torre Eiffel" (Span: [1, 2])

Score Conjunto: 93.1%

💡 **Sem Resposta (Unanswerable Questions):** Se o texto não contiver a resposta, o modelo aprende a apontar tanto o *Start* quanto o *End* para a posição 0 (o token [CLS])!



Aplicação 4: Busca Semântica e Bancos Vetoriais

A Revolução do Sentence-BERT: Como Sair de Horas para Milissegundos com Bi-Encoders

Arquiteturas de Recuperação Vetorial:

Pipeline Moderno em 2 Estágios (Indústria)

Bi-Encoder (SBERT) vs Cross-Encoder

De horas de computação para **milissegundos** com Bancos Vetoriais

ESTÁGIO 1: RECUPERAÇÃO (BI-ENCODER)

~5 ms

- **Modelos:** Sentence-BERT, BGE-large, all-MiniLM-L6.
- **Ação:** Converte Query em vetor $u \in \mathbb{R}^{768}$.
- **Busca:** Similaridade de Cosseno via ANN em 10.000.000 de documentos (FAISS, Chroma, Milvus).
- **Filtro:** Seleciona os **Top-100** candidatos mais promissores.

Top-100



ESTÁGIO 2: RERANKER (CROSS-ENCODER)

~25 ms

- **Modelo:** BERT Cross-Encoder ([CLS] Q [SEP] Doc [SEP]).
- **Ação:** Atenção cruzada profunda e bidirecional entre cada par.
- **Precisão:** Reordena os 100 documentos com precisão cirúrgica.
- **Resultado Final:** Entrega os **Top-5** com acurácia máxima ao usuário.

⚡ O Melhor dos Dois Mundos na Engenharia de IA:

Velocidade instantânea de vetores (Bi-Encoder) + Máxima capacidade semântica relacional (Cross-Encoder BERT). Latência total: **~30 ms** para milhões de arquivos!

A Árvore Genealógica das Evoluções do BERT

De 2018 aos Dias Atuais: RoBERTa, DistilBERT, ALBERT e DeBERTa

Google AI (2018)

Base Histórica

BERT

Pioneiro da Atenção Bidirecional Profunda

- Introduziu os objetivos MLM e NSP
- WordPiece com 30.522 tokens
- Treinado em 16 GB de texto (Books + Wiki)

⚡ O marco que iniciou a era dos modelos de linguagem pré-treinados.

Meta / FAIR (2019)

Performance Máxima

RoBERTa

Otimização Rigorosa de Treinamento

- Removeu completamente o NSP (provou desnecessário)
- Dynamic Masking (máscaras mudam a cada época)
- Treinado com 160 GB de texto (10× mais dados)
- Batch size gigante (8.000 sequências)

🎯 Provou que o pré-treino do BERT original era "subtreinado".

Hugging Face (2019)

Alta Velocidade & Produção

DistilBERT

Destilação de Conhecimento (Compressão)

- 6 camadas em vez de 12 (40% menor)
- 60% mais rápido em inferência na CPU
- Retém 97% da precisão do BERT original
- Padrão da indústria para microsserviços

🚀 A escolha padrão para APIs e dispositivos móveis.

Microsoft (2020-2021)

Estado da Arte (SOTA)

DeBERTa

Atenção Desacoplada (Disentangled)

- Separa vetores de conteúdo e posição relativa
- Enhanced Mask Decoder na cabeça do MLM
- Superou a linha de base humana no SuperGLUE
- Top 1 em tarefas de NLI e raciocínio textual

🧠 O ápice em precisão entre todas as redes Encoder-only.

💡 **Guia Prático para Engenheiros:** Em produção, comece com DistilBERT se latência for prioridade (10ms). Migre para RoBERTa-base se precisar de acurácia robusta. Use DeBERTa-v3 para desafios analíticos complexos.

Laboratório 4: Comparador de Modelos Encoder-Only

Simule o Consumo de VRAM, Latência de Inferência e Throughput em Função do Batch e Sequência

DistilBERT
66M • GLUE 77

BERT-Base
110M • GLUE 79.6

RoBERTa-Base
125M • GLUE 86.4

DeBERTa-v3-Base
86M • GLUE 88.1

BERT-Large
340M • GLUE 82.1

CONTROLES DE INFERÊNCIA / CARGA DE TRABALHO:

Tamanho do Batch (B):



Comprimento da Sequência (L):



VRAM ESTIMADA (FP32)

~656 MB

Pesos: 440MB + Ativações: 144.0MB

LATÊNCIA GPU (ESTIMADA)

~8.0 ms

Tempo para lote de 16

THROUGHPUT

~2000 seq/s

Vazão contínua em GPU

GLUE BENCHMARK

79.6 pts

Média de precisão geral

Recomendação de Arquitetura:

Padrão didático da literatura acadêmica e classificação geral.

Trade-off Fundamental: O custo de memória quadrático da autoatenção $O(L^2)$ torna sequências longas (512 tokens) 16× mais pesadas na camada de atenção do que frases curtas (128 tokens).

Laboratório 5: Quiz de Fixação de Conhecimentos sobre BERT

Desafie seu Conhecimento em 4 Perguntas Rigorosas de Nível Pós-Graduação

Quiz Interativo de Fixação: BERT, MLM, NSP e Arquiteturas Encoder-Only

Pergunta 1 de 4 • Pontuação: 0 / 4

1. Por que o BERT adota o Masked Language Model (MLM) em vez de modelagem autorregressiva tradicional de próximo token (como nos GPTs)?

- ☐ A Porque redes autorregressivas exigem atenção triangular causal, impossibilitando que cada token enxergue palavras passadas e futuras simultaneamente.
- ☐ B Porque o cálculo do produto escalar escalado ($Q \cdot K^T$) só converge matematicamente se ao menos 15% das entradas forem zeradas.
- ☐ C Porque o algoritmo de tokenização WordPiece não suporta leitura causal da esquerda para a direita.
- ☐ D Porque o token [CLS] só consegue agregar informação se todas as palavras do texto forem convertidas em [MASK].